

Eigen-Decomposition and SVD

Unlocking the Structure of Data

Firuz Kamalov

MAI 600: Mathematics and Statistics for AI

September 19, 2025

Why Decompose Matrices?

1. To Understand the Transformation

A matrix is a linear transformation. Decomposing it is like taking a car engine apart to see how it works. We can isolate its fundamental actions: **rotation, scaling, and projection**.

Why Decompose Matrices?

1. To Understand the Transformation

A matrix is a linear transformation. Decomposing it is like taking a car engine apart to see how it works. We can isolate its fundamental actions: **rotation, scaling, and projection**.

2. To Simplify and Approximate Data

Many complex datasets can be approximated by simpler, lower-rank matrices. This is the core idea behind:

- **Dimensionality Reduction:** Reducing the number of features while keeping the most important information.
- **Noise Reduction:** Separating the "signal" from the "noise" in data.
- **Data Compression:** Storing the essential parts of a matrix instead of the whole thing.

- 1 Eigenvectors and Diagonalization
- 2 Singular Value Decomposition (SVD)
- 3 Applications of SVD

Eigenvectors: The Stable Directions

Definition

An **eigenvector** of a square matrix **A** is a non-zero vector **v** that, when multiplied by **A**, is simply scaled by a scalar value λ . The vector's direction does not change.

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

The scalar λ is the **eigenvalue** corresponding to that eigenvector.

Example

Consider the non-symmetric matrix $\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$ and the vector $\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Then

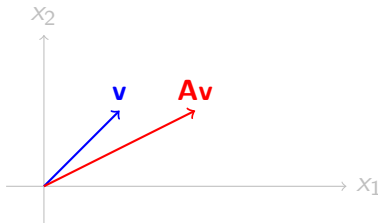
$$\mathbf{A}\mathbf{v} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5 \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Thus, **v** is an eigenvector and $\lambda = 5$ is the corresponding eigenvalue of **A**.

Geometric Intuition

Imagine a linear transformation squishing and rotating the entire space. Eigenvectors are the special vectors that **stay on their own span**. They don't get rotated off their line; they only get stretched or shrunk.

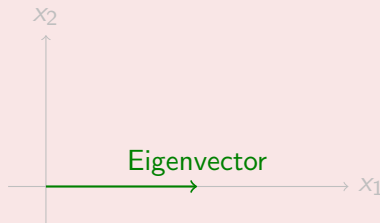
Consider a shear transformation $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$. Most vectors get knocked off their line.



The Eigenvector

The vector $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ is an eigenvector of this matrix. Notice it is only scaled (not even stretched in this case, as $\lambda = 1$).

$$\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$



Problem

Find the eigenvalues for the matrix $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

Hint: The eigenvalues λ are the solutions to the characteristic equation $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

Solution

1. **Set up the characteristic equation:**

$$\begin{aligned}\det(\mathbf{A} - \lambda \mathbf{I}) &= \det \left(\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \\ &= \det \begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = 0\end{aligned}$$

Solution

1. **Set up the characteristic equation:**

$$\begin{aligned}\det(\mathbf{A} - \lambda\mathbf{I}) &= \det\left(\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) \\ &= \det\begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = 0\end{aligned}$$

2. **Calculate the determinant:**

$$\begin{aligned}(2 - \lambda)(2 - \lambda) - (1)(1) &= 0 \\ \lambda^2 - 4\lambda + 3 &= 0\end{aligned}$$

Solution

1. **Set up the characteristic equation:**

$$\begin{aligned}\det(\mathbf{A} - \lambda\mathbf{I}) &= \det\left(\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) \\ &= \det\begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = 0\end{aligned}$$

2. **Calculate the determinant:**

$$\begin{aligned}(2 - \lambda)(2 - \lambda) - (1)(1) &= 0 \\ \lambda^2 - 4\lambda + 3 &= 0\end{aligned}$$

3. **Solve the quadratic equation for λ :**

$$(\lambda - 3)(\lambda - 1) = 0$$

The eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 1$.

The Big Idea

If a square matrix $\mathbf{A}$ has enough linearly independent eigenvectors, we can use them as a new basis for our space. In this new basis, the transformation $\mathbf{A}$ acts simply as a scaling operation.

The Decomposition

This leads to the decomposition:

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$$

- $\mathbf{P}$ is a matrix whose columns are the eigenvectors of $\mathbf{A}$.
- $\mathbf{D}$ is a diagonal matrix containing the corresponding eigenvalues.
- $\mathbf{P}^{-1}$ is the inverse of $\mathbf{P}$.

Diagonalization Example: Putting it all Together

Recall from previous slide

For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, we found eigenvalues $\lambda_1 = 3$ and $\lambda_2 = 1$.

Recall from previous slide

For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, we found eigenvalues $\lambda_1 = 3$ and $\lambda_2 = 1$.

① Find eigenvectors for each eigenvalue:

- For $\lambda_1 = 3$: Solve $(\mathbf{A} - 3\mathbf{I})\mathbf{x} = \mathbf{0}$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow -x_1 + x_2 = 0 \Rightarrow x_1 = x_2$$

$$\text{Let } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Recall from previous slide

For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, we found eigenvalues $\lambda_1 = 3$ and $\lambda_2 = 1$.

① Find eigenvectors for each eigenvalue:

- For $\lambda_1 = 3$: Solve $(\mathbf{A} - 3\mathbf{I})\mathbf{x} = \mathbf{0}$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow -x_1 + x_2 = 0 \Rightarrow x_1 = x_2$$

$$\text{Let } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

- For $\lambda_2 = 1$: Solve $(\mathbf{A} - 1\mathbf{I})\mathbf{x} = \mathbf{0}$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow x_1 + x_2 = 0 \Rightarrow x_1 = -x_2$$

$$\text{Let } \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

1 Form the matrices **P** and **D**:

- The eigenvector matrix **P** (columns are eigenvectors):

$$\mathbf{P} = [\mathbf{v}_1 \quad \mathbf{v}_2] = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

1 Form the matrices **P** and **D**:

- The eigenvector matrix **P** (columns are eigenvectors):

$$\mathbf{P} = [\mathbf{v}_1 \quad \mathbf{v}_2] = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

- The eigenvalue matrix **D** (diagonal with eigenvalues):

$$\mathbf{D} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

1 Form the matrices **P** and **D**:

- The eigenvector matrix **P** (columns are eigenvectors):

$$\mathbf{P} = [\mathbf{v}_1 \quad \mathbf{v}_2] = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

- The eigenvalue matrix **D** (diagonal with eigenvalues):

$$\mathbf{D} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

Thus, the diagonalization of **A** is $\mathbf{A} = \mathbf{PDP}^{-1}$, where **P** and **D** are as shown.

What does it mean?

Applying the transformation $\mathbf{A}$ is equivalent to:

- 1 $\mathbf{P}^{-1}$: Change from the standard basis to the eigenvector basis.
- 2 $\mathbf{D}$: Scale along the new basis axes (a very simple operation).
- 3 $\mathbf{P}$: Change back to the standard basis.

The Limitation of Eigendecomposition

It only works for square matrices. But most real-world data is not square! (e.g., user-item matrices, document-term matrices).

SVD: The Generalization of Eigendecomposition

The Limitation of Eigendecomposition

It only works for square matrices. But most real-world data is not square! (e.g., user-item matrices, document-term matrices).

Singular Value Decomposition (SVD)

SVD is a powerful factorization that works for **any** $m \times n$ matrix **A**. It breaks the matrix into three components:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- **U**: An $m \times m$ orthogonal matrix. Its columns are the **left-singular vectors**.
- **Σ** : An $m \times n$ diagonal matrix. Its diagonal entries are the **singular values**.
- **V^T** : An $n \times n$ orthogonal matrix (so its transpose is its inverse). Its rows are related to the **right-singular vectors**.

SVD states that any linear transformation can be broken down into a sequence of three fundamental geometric operations:

- ① **Rotation ($\mathbf{V}^T$):** The right-singular vectors in $\mathbf{V}$ form an orthonormal basis in the input space. $\mathbf{V}^T$ rotates the input data to align with these axes.

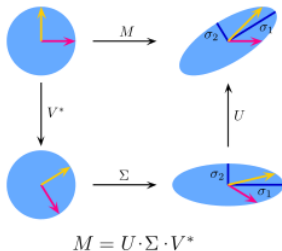
SVD states that any linear transformation can be broken down into a sequence of three fundamental geometric operations:

- ① **Rotation ($\mathbf{V}^T$):** The right-singular vectors in $\mathbf{V}$ form an orthonormal basis in the input space. $\mathbf{V}^T$ rotates the input data to align with these axes.
- ② **Scaling ($\mathbf{\Sigma}$):** The singular values $(\sigma_1, \sigma_2, \dots)$ on the diagonal of $\mathbf{\Sigma}$ scale the data along the new axes.

SVD: Geometric Intuition

SVD states that any linear transformation can be broken down into a sequence of three fundamental geometric operations:

- 1 **Rotation ($\mathbf{V}^T$):** The right-singular vectors in $\mathbf{V}$ form an orthonormal basis in the input space. $\mathbf{V}^T$ rotates the input data to align with these axes.
- 2 **Scaling ($\mathbf{\Sigma}$):** The singular values ($\sigma_1, \sigma_2, \dots$) on the diagonal of $\mathbf{\Sigma}$ scale the data along the new axes.
- 3 **Another Rotation ($\mathbf{U}$):** The left-singular vectors in $\mathbf{U}$ form an orthonormal basis in the output space. $\mathbf{U}$ rotates the scaled data into the final position.



The Power of Truncated SVD

The singular values in $\mathbf{\Sigma}$ are always arranged in decreasing order: $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq 0$. They represent the "importance" of each dimension.

Low-Rank Approximation

The most powerful application of SVD comes from the fact that we can create an approximation of $\mathbf{A}$ by keeping only the first k singular values. This is called the **Truncated SVD**.

$$\mathbf{A} \approx \mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$$

This creates the best possible rank- k approximation of the original matrix $\mathbf{A}$.

The Power of Truncated SVD

The singular values in $\mathbf{\Sigma}$ are always arranged in decreasing order: $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq 0$. They represent the "importance" of each dimension.

Low-Rank Approximation

The most powerful application of SVD comes from the fact that we can create an approximation of $\mathbf{A}$ by keeping only the first k singular values. This is called the **Truncated SVD**.

$$\mathbf{A} \approx \mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$$

This creates the best possible rank- k approximation of the original matrix $\mathbf{A}$.

Why is this useful?

If our original data matrix has 1000 features (columns), but SVD shows that the first 50 singular values contain 99% of the information (variance), we can use the truncated SVD to represent our data in a 50-dimensional space with minimal loss of information. This is the core of dimensionality reduction.

Application 1: Image Compression

An image is just a matrix of pixel values. A grayscale image is one matrix, while a color image can be represented by three matrices (Red, Green, Blue).

How it Works

- 1 Treat the image as a matrix $\mathbf{A}$.
- 2 Compute the SVD: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
- 3 Keep only the top k singular values to create an approximate image $\mathbf{A}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T$.
- 4 To store the compressed image, we only need to store $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, and $\mathbf{V}_k^T$, which requires much less space than the original $\mathbf{A}$.

Example

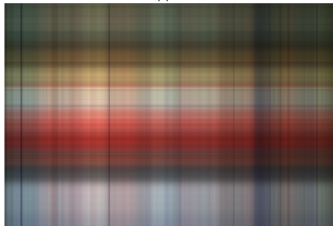
<https://timbaumann.info/svd-image-compression-demo/>

Application 1: Image Compression

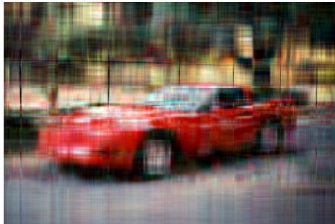
Original



Rank-1 Approximation



Rank-10 Approximation



Rank-100 Approximation



Application 2: Dimensionality Reduction (PCA)

Principal Component Analysis (PCA)

PCA is one of the most widely used techniques for dimensionality reduction in AI. Its goal is to find the "principal components" (directions of greatest variance) in the data and project the data onto them.

SVD is the Engine of PCA

The SVD of a data matrix provides the principal components directly.

- The **right-singular vectors** ($\mathbf{V}$) give you the principal components (the new axes).
- The **singular values** ($\mathbf{\Sigma}$) tell you the amount of variance captured by each component.

By keeping only the top k vectors from $\mathbf{V}$, we can project our high-dimensional data into a k -dimensional space, which is easier to work with for machine learning algorithms.

Application 3: Recommender Systems

The User-Item Matrix

Recommender systems often start with a huge, sparse matrix where rows are users, columns are items (e.g., movies), and the entries are user ratings. The goal is to predict the missing ratings.

SVD for Latent Factor Models

SVD can decompose this matrix to uncover **latent factors**—hidden concepts that explain the ratings.

$$\text{Ratings} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$$

- The rows of $\mathbf{U}_k$ represent users in a "latent factor" space. For movies, these factors might correspond to genres like "comedy" vs. "action" or "serious" vs. "lighthearted".
- The columns of $\mathbf{V}_k^T$ represent items in the same latent factor space.

To predict a rating for a user and an item, we can take the dot product of that user's vector

Application 4: Natural Language Processing (NLP)

Term-Document Matrix

In NLP, we often represent a collection of documents with a matrix where rows are unique words (terms) and columns are documents. The entries can be word counts or frequencies.

Latent Semantic Analysis (LSA)

SVD is the core of a technique called Latent Semantic Analysis (LSA).

- By performing a truncated SVD on the term-document matrix, LSA can uncover the "topics" or "concepts" that are present across the documents.
- It helps group documents that are semantically similar, even if they don't share the exact same words. For example, it can learn that "automobile" and "car" are related because they appear in similar document contexts.
- This is used for document clustering, information retrieval, and improving search engine results.

Problem

You are given a user-movie rating matrix of size 1,000,000 users $\times$ 5,000 movies. You compute its SVD.

- 1 What are the dimensions of the $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{V}^T$ matrices?
- 2 You find that the first 100 singular values are much larger than the rest. You decide to use a truncated SVD with $k = 100$ to build a recommender system. How does this help you save memory when storing the model?

Solution

① Dimensions of the full SVD matrices:

- $\mathbf{A}$ is $1,000,000 \times 5,000$.
- $\mathbf{U}$ is $1,000,000 \times 1,000,000$.
- $\mathbf{\Sigma}$ is $1,000,000 \times 5,000$.
- $\mathbf{V}^T$ is $5,000 \times 5,000$.

Solution

① Dimensions of the full SVD matrices:

- $\mathbf{A}$ is $1,000,000 \times 5,000$.
- $\mathbf{U}$ is $1,000,000 \times 1,000,000$.
- $\mathbf{\Sigma}$ is $1,000,000 \times 5,000$.
- $\mathbf{V}^T$ is $5,000 \times 5,000$.

② Memory savings with Truncated SVD ($k = 100$):

- Storing the original dense matrix would require $1,000,000 \times 5,000 = 5 \times 10^9$ numbers.
- With truncated SVD, we store $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, and $\mathbf{V}_k^T$:
 - $\mathbf{U}_k$: $1,000,000 \times 100$
 - $\mathbf{\Sigma}_k$: 100×100 (or just 100 numbers for the diagonal)
 - $\mathbf{V}_k^T$: $100 \times 5,000$
- Total numbers to store: $(10^6 \times 100) + 100 + (100 \times 5000) \approx 10^8 + 5 \times 10^5 \approx 1.005 \times 10^8$.
- This is a massive reduction from the original 5×10^9 elements, making it feasible to store and work with the latent factor model.

- **Eigen-Decomposition** explains a square matrix transformation in terms of its stable directions (eigenvectors) and scaling factors (eigenvalues). It's a change of basis to where the transformation is simple.
- **Singular Value Decomposition (SVD)** is the general-purpose tool. It decomposes *any* matrix into rotation, scaling, and another rotation.
- **Truncated SVD** is the key to practical applications. By keeping only the most important singular values, we can create low-rank approximations of our data.
- **AI Applications** are widespread:
 - **Image Compression:** Reducing storage size.
 - **PCA:** Finding the most important features in data.
 - **Recommender Systems:** Uncovering latent user tastes.
 - **NLP:** Discovering topics and semantic meaning in text.

Questions?